

Challenge 02: Execução de Testes da API ServeRest

1. Identificação do Projeto

Nome do Projeto:

Execução e Monitoramento de Testes Funcionais da API ServeRest

API Testada:

ServeRest – Compass UOL (ServeRest)

Versão do Plano:

1.0

Data de Criação:

10/02/2026

Data da Última Atualização:

13/02/2026

Responsável pela Elaboração:

Eduardo Neves de Souza

Responsável pela Aprovação:

Avaliadores do Programa de Bolsas

Tempo Estimado de Execução:

Em torno de 16 horas (considerando o tempo integral do estágio durante a semana)

2. Escopo

Este documento contempla a execução dos casos de teste previamente definidos no Plano de Testes versão 1.0, com foco na validação funcional da API ServeRest e no registro estruturado de defeitos identificados durante a execução.

A execução abrange:

- Testes funcionais positivos (fluxos esperados)
- Testes funcionais negativos (validação de regras de negócio)
- Testes de contrato (validação de estrutura de request e response)
- Testes exploratórios orientados por cenários de quebra de fluxo

Estão incluídos no escopo os seguintes recursos da API:

Usuários

Validação de criação, atualização, exclusão e regras de unicidade de e-mail.

Autenticação

Validação de geração de token e comportamento em cenários de credenciais inválidas ou ausência de payload.

Produtos

Validação de permissões administrativas, unicidade de nome, integridade de payload e restrição de exclusão.

Carrinhos

Validação de criação, conclusão, cancelamento, dependência entre usuário e produto e atualização de estoque.

Ficam fora do escopo:

- Testes de performance
- Testes de segurança avançados
- Testes de carga
- Testes de integração com sistemas externos

3. Objetivos

3.1 Objetivo Geral

Validar o comportamento real da API ServeRest por meio da execução prática dos cenários planejados, garantindo aderência às regras de negócio, consistência de respostas e tratamento adequado de erros.

3.2 Objetivos Específicos

- Confirmar que os status codes retornados estão alinhados com o esperado.
- Verificar se as regras de negócio estão sendo corretamente aplicadas.
- Identificar falhas funcionais que comprometam integridade de dados.
- Detectar inconsistências no tratamento de erros.
- Validar a conformidade estrutural das respostas da API.

4. Estratégia de Execução

Estratégia adotada: Execução estruturada + exploração orientada

A execução foi realizada com base nos casos CT-F01 a CT-F20 previamente definidos, seguindo a priorização estabelecida no plano.

Após a execução roteirizada, foram realizadas sessões adicionais de exploração com foco em:

- Quebra de fluxo
- Reutilização de recursos
- Manipulação de payload
- Testes de idempotência
- Validação de associação entre token e recurso

A abordagem buscou não apenas validar o “caminho feliz”, mas provocar a API em cenários críticos.

5. Estratégia de Execução

5.1 Cenários Executados

Foram executados **10 dos 20 cenários planejados**, priorizando cenários de prioridade Alta conforme estratégia de mitigação de risco do produto.

Prioridade	Planejados	Executados	Passou	Falhou/Bug
Crítica	0	0	0	0
Alta	14	9	6	3
Média	6	1	1	0
Baixa	0	0	0	0
TOTAL	20	10	5	5

5.2 Resultados Gerais

Cenários que passaram: **5 (50% dos executados)**

Cenários com bugs: **5 (50% dos executados)**

Bugs críticos encontrados: **0**

Bugs altos encontrados: **2**

5.3 Tabela de Cenários Executados

ID	Cenário	Prioridade	Status	Resultado	Bug
CT-F01	Registrar usuário	Alta	Executado	Passou	-

	com dados válidos				
CT-F02	Impedir cadastro com e-mail já existente	Alta	Executado	Passou	-
CT-F03	Rejeitar cadastro sem campos obrigatórios	Alta	Executado	Falhou	BUG-01
CT-F04	Validar schema de cadastro	Média	Executado	Passou	-
CT-F05	Autenticar usuário com credenciais válidas	Alta	Executado	Passou	-
CT-F06	Bloquear login com credenciais inválidas	Alta	Executado	Falhou	BUG-01, BUG-02
CT-F09	Criar produto como administrador	Alta	Executado	Passou	-
CT-F11	Rejeitar criação de produto com	Média	Executado	Falhou	BUG-04

	dados inválidos				
CT-F14	Criar carrinho com produtos inválidos	Alta	Executado	Falhou	BUG-04, BUG-05
CT-F18	Concluir carrinho existente com sucesso	Alta	Executado	Falhou	-

6. Bugs Encontrados

BUG 01: Aceitação de Valores Vazios

Título: API permite criação de recurso com campos obrigatórios contendo valor vazio

Descrição:

A API aceita o valor literal vazio em campos que deveriam ser obrigatórios para a integridade do objeto, como nome de usuário ou descrição de produto. Ao permitir vazio, o sistema persiste dados incompletos, podendo gerar falhas de renderização, inconsistências de regra de negócio e erros em integrações futuras.

Endpoints afetados:

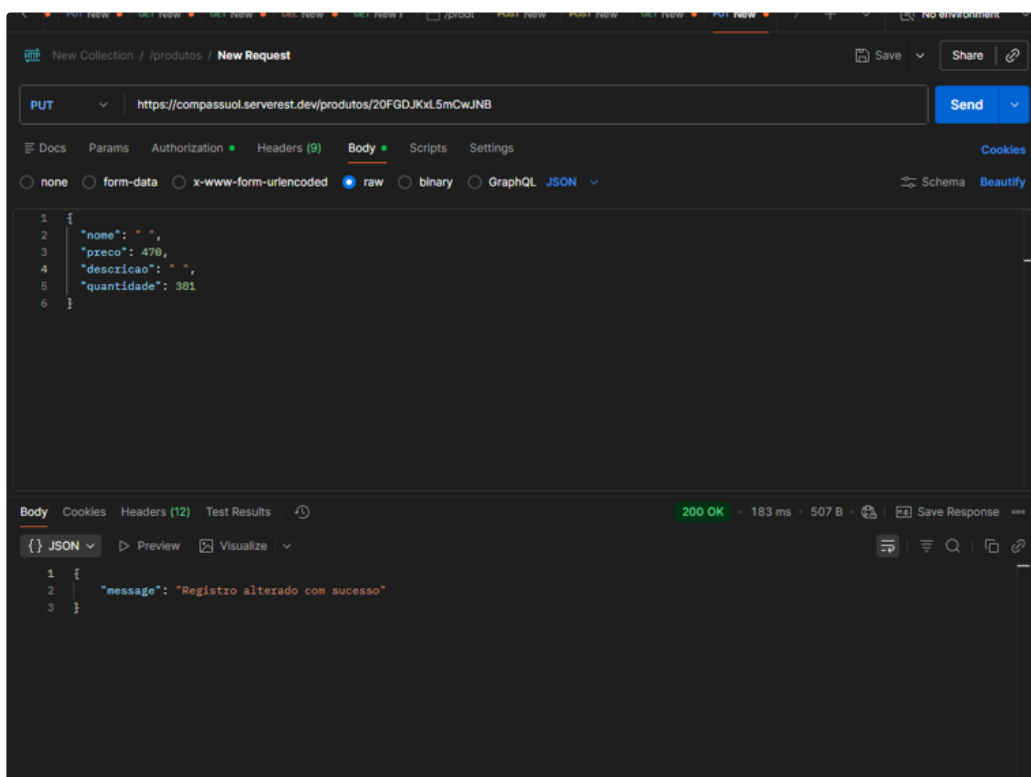
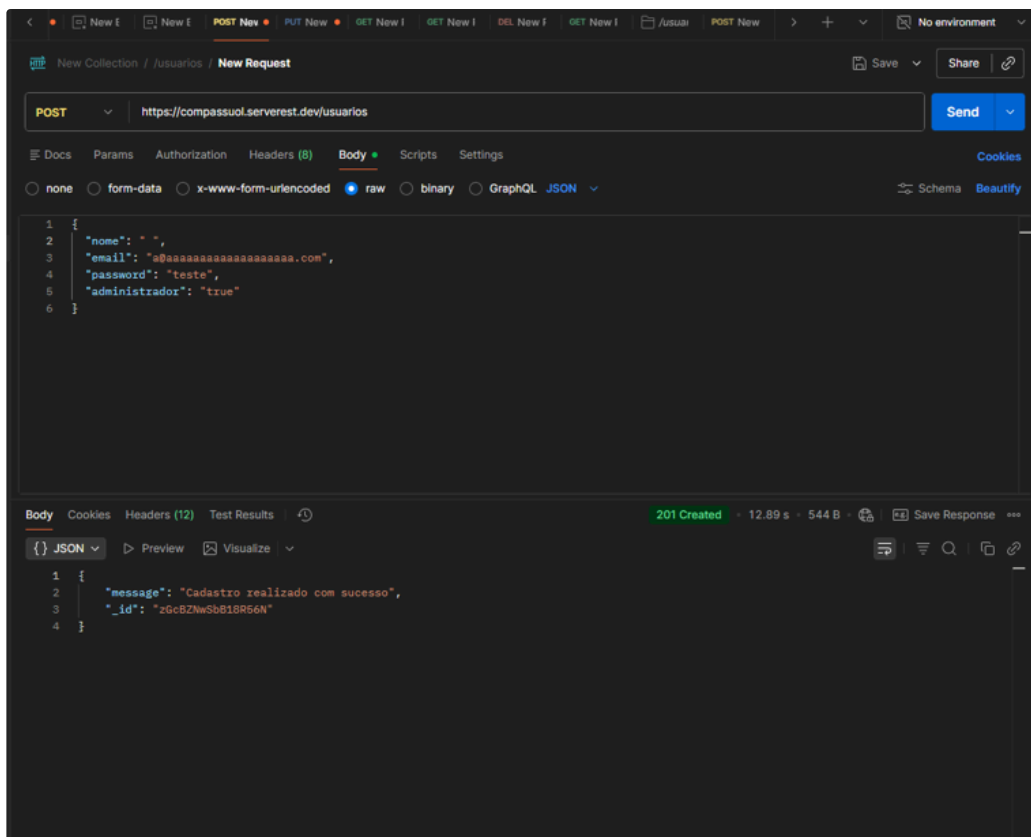
POST /usuarios

POST /produtos

Passos para reprodução:

1. Realizar um POST para /usuarios ou /produtos.
2. No corpo da requisição, definir um campo obrigatório como vazio.
3. Enviar a requisição.
4. Verificar que o retorno é 201 Created.

Evidências:



Resultado atual:

A API retorna 201 Created e persiste o dado com campo vazio.

Resultado esperado:

A API deve retornar 400 Bad Request informando que o campo obrigatório não pode ser nulo.

Impacto: Alto. Compromete a integridade dos dados e pode causar falhas sistêmicas.

Prioridade: Alta.

BUG 02: Falha na Sanitização de Inputs

Título: API permite cadastro com nomes compostos por caracteres especiais

Descrição:

A API não valida adequadamente o conteúdo textual enviado. É possível cadastrar usuários ou produtos com nomes contendo apenas caracteres especiais, caracterizando falha na validação de regra de negócio e comprometendo a qualidade dos dados armazenados.

Endpoints afetados:

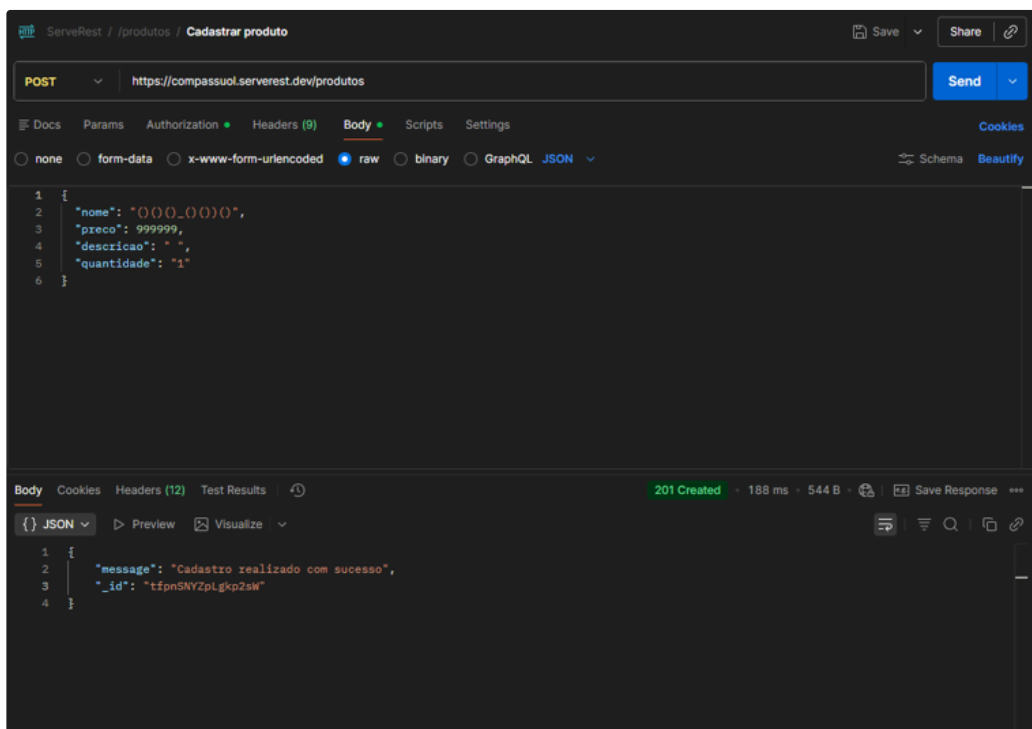
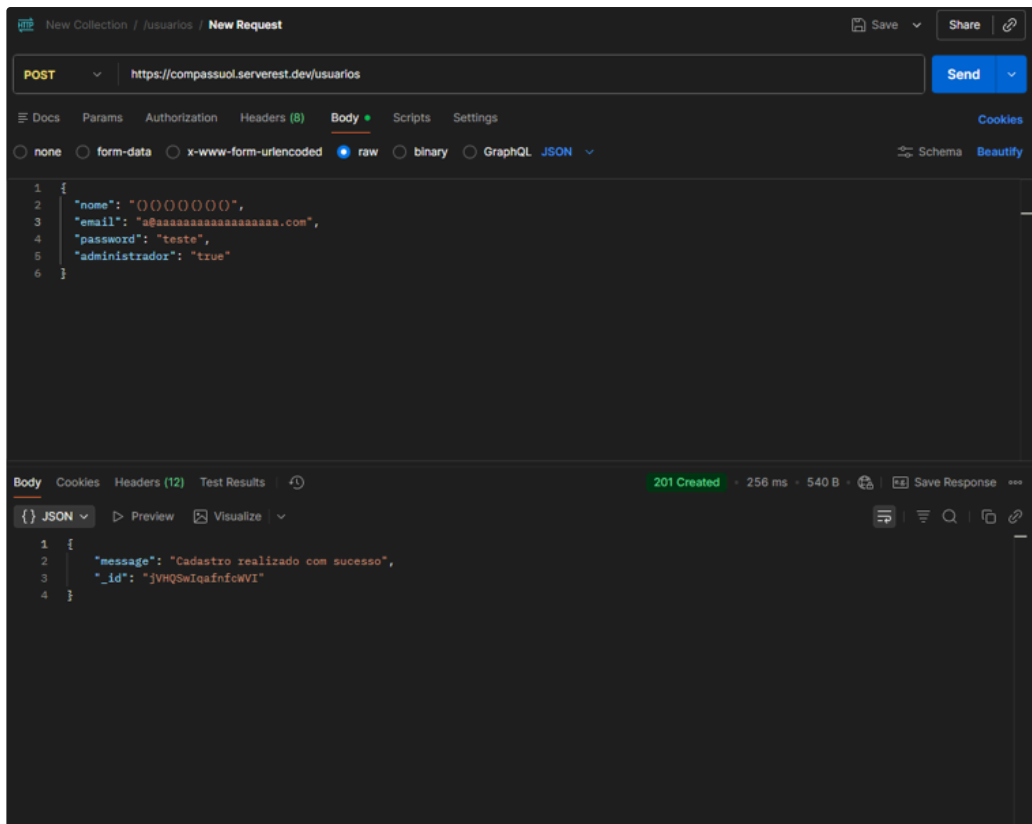
POST /usuarios

PUT /produtos

Passos para reprodução:

1. Realizar um POST ou PUT para /usuarios ou para /produtos.
2. No campo nome, inserir apenas espaços ou caracteres especiais.
3. Enviar a requisição.
4. Observar retorno 201 Created ou 200 OK.

Evidências:



Resultado atual:

A API aceita e persiste o valor inválido.

Resultado esperado:

O sistema deve rejeitar nomes que não contenham ao menos um caractere alfanumérico válido, retornando 400 Bad Request.

Impacto: Médio. Afeta consistência e confiabilidade dos dados.

Prioridade: Média.

BUG 03: Inconsistência de Status Code no DELETE

Título: API retorna 200 OK para recurso não encontrado em operação DELETE

Descrição:

Ao tentar excluir um recurso utilizando um ID inexistente, a API retorna 200 OK com mensagem informando que nenhum registro foi excluído. O status 200 indica sucesso na operação, o que não está semanticamente alinhado com a inexistência do recurso.

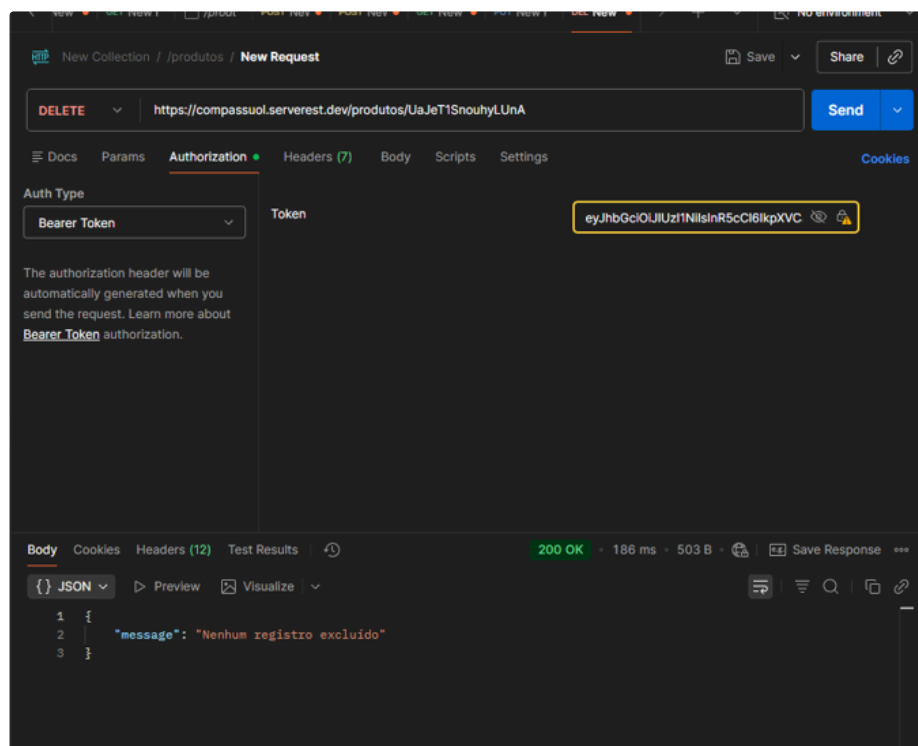
Endpoint afetado:

DELETE /produtos/{id} ou DELETE/produtos/{id}

Passos para reprodução:

1. Realizar um DELETE para /produtos/{id_inexistente}.
2. Observar o status code e o corpo da resposta.

Evidências:



Resultado atual:

Retorno 200 OK com mensagem informando que nenhum registro foi excluído.

Resultado esperado:

Retorno 404 Not Found indicando que o recurso não foi localizado.

Impacto: Baixo. Não impede o funcionamento do sistema, mas compromete a padronização REST e a semântica correta de status HTTP.

Prioridade: Baixa.

BUG 04: Falha de Validação de Tipagem

Título: API aceita valores String em campos definidos como numéricos

Descrição:

A API aceita valores enviados como String em campos que deveriam ser numéricos, como quantidade ou preço. Isso demonstra ausência de validação de tipagem no recebimento do JSON, podendo causar inconsistências em cálculos ou integrações futuras.

Endpoints afetados:

POST /produtos

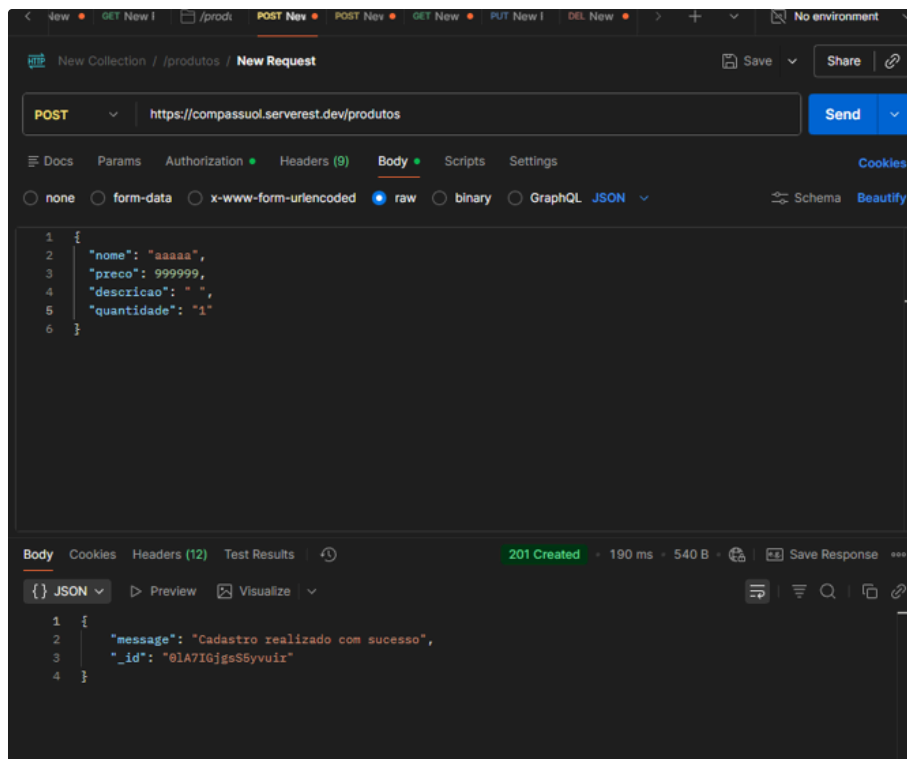
POST /carrinhos

POST /usuarios

Passos para reprodução:

1. Realizar um POST para /produtos ou /carrinhos.
2. Enviar valores numéricos entre aspas nos campos quantidade ou preço.
3. Observar que a requisição é processada normalmente.

Evidências:



Resultado atual:

A API aceita e processa o valor sem validação de tipo.

Resultado esperado:

A API deve validar o tipo do dado e retornar 400 Bad Request caso o valor não seja numérico no formato correto.

Impacto: Médio. Pode comprometer operações matemáticas e integridade de dados.

Prioridade: Média.

BUG 05: Falha de Integridade na Persistência de Carrinho

Título: API permite criação de carrinho contendo produtos inválidos

Descrição:

A API permite a criação de um carrinho mesmo quando os produtos informados possuem dados inválidos ou inconsistentes. Isso caracteriza falha de validação na camada de persistência e compromete a integridade dos dados armazenados.

Ao permitir que um carrinho seja salvo com produtos inválidos, o sistema pode gerar inconsistências em consultas futuras, falhas no processamento de pedidos e erros em cálculos de valores totais.

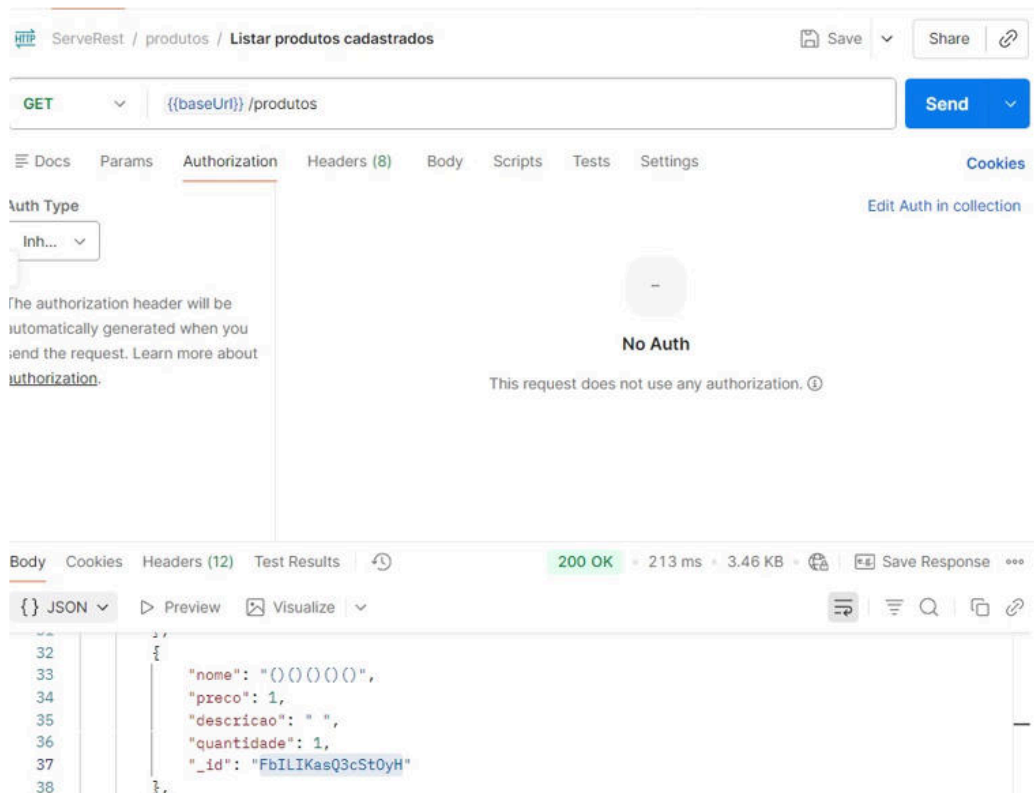
Endpoint afetado:

POST /carrinhos

Passos para reprodução:

1. Realizar um POST para /carrinhos.
2. Informar no corpo da requisição um ou mais produtos com dados inválidos.
3. Enviar a requisição.
4. Observar que o carrinho é criado com status 201 Created.

Evidências:



ServeRest / produtos / Listar produtos cadastrados

GET {{baseUrl}} /produtos

Send

Docs Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Auth Type

Inh...

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#).

No Auth

This request does not use any authorization. ⓘ

Body Cookies Headers (12) Test Results

200 OK · 213 ms · 3.46 KB

Save Response

{ } JSON Preview Visualize

```
105      "descricao": " ",
106      "quantidade": 1,
107      "_id": "xAAVKRczGjFjuqeB"
108    },
109    {
110      "nome": "null",
111      "preco": 1,
112      "descricao": " ",
113      "quantidade": 1,
114      "_id": "xZ05nKwX26rro3yT"
115    }
116  ]
117 }
```

GET Listar POST Cad DEL Exclu GET Listar PUT Editar GET Busca

ServeRest / carrinhos / Cadastrar carrinho

POST {{baseUrl}} /carrinhos

Send

Docs Params Authorization Headers (12) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "produtos": [
3     {
4       "idProduto": "xZ05nKwX26rro3yT",
5       "quantidade": 1
6     },
7     {
8       "idProduto": "FbILIKasQ3cSt0yH",
9       "quantidade": 1
10    }
11  ]
12 }
```

Body Cookies Headers (12) Test Results

201 Created · 234 ms · 544 B

Save Response

{ } JSON Preview Visualize

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "GKd4jpZznJdQ2r2m"
4 }
```

Resultado atual:

A API cria e persiste o carrinho mesmo contendo produtos inválidos.

Resultado esperado:

A API deve validar previamente os dados dos produtos informados. Caso algum produto esteja inválido, a requisição deve ser rejeitada com 400 Bad Request.

Impacto:

Alto. Compromete integridade dos dados e pode gerar falhas em fluxos críticos como fechamento de pedido e faturamento.

Prioridade: Alta.

7. Testes Automatizados

Os testes automatizados foram implementados para validar automaticamente o comportamento da API após cada requisição realizada.

Em cada chamada, são verificados o status code HTTP, a mensagem retornada no corpo da resposta e a estrutura do JSON, garantindo que estejam de acordo com o contrato esperado. Foram contemplados cenários positivos, como cadastro com dados válidos (esperando 201 Created), e cenários negativos, como tentativa de cadastro com e-mail duplicado (esperando 400 Bad Request), além de cenários de contrato.

A execução automática permite identificar rapidamente inconsistências entre o comportamento esperado e o comportamento real da API, além de reduzir risco de regressão a cada nova alteração no sistema.

Postman interface showing a successful POST request to `https://compassuol.serverest.dev/usuarios`. The request body is a JSON object:

```
1 {
2   "nome": "O O O O O O O O",
3   "email": "o@ooooooooooooo.com",
4   "password": "teste",
5   "administrador": "true"
6 }
```

The response status is **201 Created** (188 ms, 544 B). The test results show three passed assertions:

- PASSED** 201 e usuário foi criado
- PASSED** Resposta com mensagem de sucesso
- PASSED** O contrato do JSON é válido

Postman interface showing a failed POST request to `https://compassuol.serverest.dev/usuarios`. The request body is a JSON object (same as above). The response status is **400 Bad Request** (508 ms, 519 B). The test results show two passed assertions:

- PASSED** Status code é 400 ao usar e-mail duplicado
- PASSED** Valida mensagem de erro de e-mail já cadastrado

The Scripts tab is active, showing the following test scripts:

```
1 pm.test("Status code é 400 ao usar e-mail duplicado", function () {
2   pm.response.to.have.status(400);
3 });
4
5 pm.test("Valida mensagem de erro de e-mail já cadastrado", function () {
6   const jsonData = pm.response.json();
7   pm.expect(jsonData.message).to.eql("Este email já está sendo usado");
8 });
```